

Том 2. Глава 5. За кулисами сингулярности: Векторы и Гравитация

В пятой главе первого тома Навигатор столкнулся с пространственным капканом: лазерный луч затачивало в черную дыру по спирали. Из этой главы ты узнаешь, как устроен графический движок, который нарисовал эту траекторию. Мы разберем, как с помощью языка Rust смоделировать искривление пространства.

1. Кирпичики трехмерного мира: `struct Vector3`

Чтобы компьютер мог работать с точками в пространстве или направлениями лазера, нам нужен математический контейнер. В Rust мы создаем его с помощью структуры (`struct`):

```
#[derive(Debug, Clone, Copy)]
struct Vector3 {
    x: f64,
    y: f64,
    z: f64,
}
```

- **Что это значит?** Мы объединили три числа с плавающей точкой повышенной точности (`f64`) под одним именем. Переменная такого типа может означать либо координату навигатора, либо силу притяжения черной дыры.
- **Зачем нужны `derive`?** Это автоматические макросы Rust. Благодаря им мы можем легко копировать наши векторы в памяти (`Clone`, `Copy`) и выводить их значения на экран для проверки (`Debug`).

2. Магия Скалярного Произведения: Метод `dot`

Как узнать, куда направлен вектор относительно другого? Для этого программисты используют скалярное произведение (`dot product`). В коде мы добавили его как метод для нашей структуры:

```
impl Vector3 {
    fn dot(&self, other: &Vector3) -> f64 {
        self.x * other.x + self.y * other.y + self.z * other.z
    }
}
```

Аналогия для навигатора: Представь, что ты светишь фонариком.

- Если метод `dot` возвращает **положительное число**, твой фонарик светит примерно в ту же сторону, куда направлен взгляд твоего друга.

- Если **ноль** — твои лучи пересекаются строго под прямым углом (90°).
- Если **отрицательное число** — ты светишь другу прямо в спину.

3. Численное интегрирование: Как двигать свет по шагам

В обычной физике свет летит по идеальной прямой. Но рядом с черной дырой прямых путей нет. Так как компьютер не умеет считать «плавное» искривление пространства целиком, мы используем **численные методы**. Мы заставляем луч лететь крошечными рывками — шагами (`STEP_SIZE = 0.1`).

Внутри главного цикла программы на каждом шаге происходят три вещи:

1. Вычисляется расстояние до сингулярности:

```
let dist_to_center = current_pos.length();
```

Если расстояние меньше безопасного предела, луч считается погибшим, и симуляция возвращает `SimulationResult::CapturedByBlackHole`.

2. Рассчитывается гравитационный пинок: Черная дыра находится в координатах `(0, 0, 0)`. Наш код вычисляет направление притяжения и изменяет вектор движения лазера:

```
let gravity_strength = G_MASS / dist_to_center.powi(2);  
direction.x += gravity_pull.x * gravity_strength * STEP_SIZE;
```

Это закон всемирного тяготения Ньютона: чем ближе луч к центру, тем сильнее меняется его направление.

3. Проверяется столкновение с обшивкой бублика: На каждом шаге мы подставляем новые координаты луча в уравнение торуса:

```
let torus_equation = (R_BIG - proj_xy).powi(2) + current_pos.z.powi(2);
```

Если результат равен радиусу трубы в квадрате, значит, луч успешно пробил пространство насквозь и врезался в противоположную стену станции!

4. Защита от бесконечного цикла (Зависания)

Если бы мы запустили луч мимо черной дыры, но силы гравитации не хватило бы для падения, он мог бы вечно кружить вокруг нее. Компьютер попал бы в бесконечный цикл, а программа бы «зависла» (состояние дедлока).

Чтобы этого не произошло, наш цикл жестко ограничен:

```
for _step in 0..500 { ... }
```

Если за 500 шагов лазер не врезался в бублик и не упал в дыру, алгоритм принудительно останавливает расчет, сообщая, что этот путь ведет «в никуда».

Задание на «Триумф» 🚀

Открой файл исходного кода `main.rs` текущей главы. Измени значение константы гравитационной массы черной дыры `G_MASS` с `150.0` на `0.0` (как будто черная дыра внезапно исчезла). Скомпилируй и запусти проект в терминале:

```
cargo run --bin chapter_5_blackhole
```

Посмотри на сгенерированный лог. Удалось ли лучу теперь долететь до противоположной стороны бублика, минуя капкан сингулярности? Убедись, что программа завершилась заветным статусом подтверждения.